

UNIVERSITAT POLITÈCNICA DE CATALUNYA

School of Professional & Executive Development

Postgrado en

Mobile Apps Design

SERVICES

Juan Luis Gorricho

SERVICES

- The main purpose of a service is to execute a task in background, without user intervention (no user interface involved).
- Nevertheless, the code of the service is executed by the main thread of the app (the UI thread); hence, if the code is computationally intensive it must be delegated to another thread: **Thread**, **AsyncTask** for an individual task or a **Timer** (jointly with a **TimerTask)** if we want to perform a repeated task.
- To execute a task in the future, repeatedly or not, aside from the app. cycle of life, we will have to program an **alarm**.

SERVICES

- What are services useful for?
  - Performing operations that need to continue even if the user leaves the application's activities, like a long download (**AsyncTask or IntentService**) (as seen with the Google Play Store) or playing music (**Foreground**) (as seen with Android music apps.).
  - Performing operations that need to exist regardless of activities coming and going, such as maintaining a chat connection (**Push service**) in support of a chat application.
  - Providing a local API to remote APIs (**bindService**), such as might be provided by a Web service, as you want an *asynchronous* behavior. The service can use AsyncTask to make any remote call.
  - Performing periodic work (**Alarm + WakefulBroadcastReceiver + IntentService**) without user intervention, akin to cron jobs or Windows scheduled tasks.

SERVICE START

- An open question is when to start a service, there are different alternatives:
  - From an activity.
  - From the application.
  - As a broadcast event.
  - Scheduling an alarm.

SERVICE ACTIVATION

- A service is started, or it is given something new to do, executing: **startService**(intent). Android executes **onStartCommand**(Intent intent, int flags, int startId) of the service.
- A service can also be bound using **bindService**(intent, serviceConnection, Context.BIND_AUTO_CREATE). Android executes **onBind**(intent) of the service. Once the connection is established, the public methods of the service can be invoked.

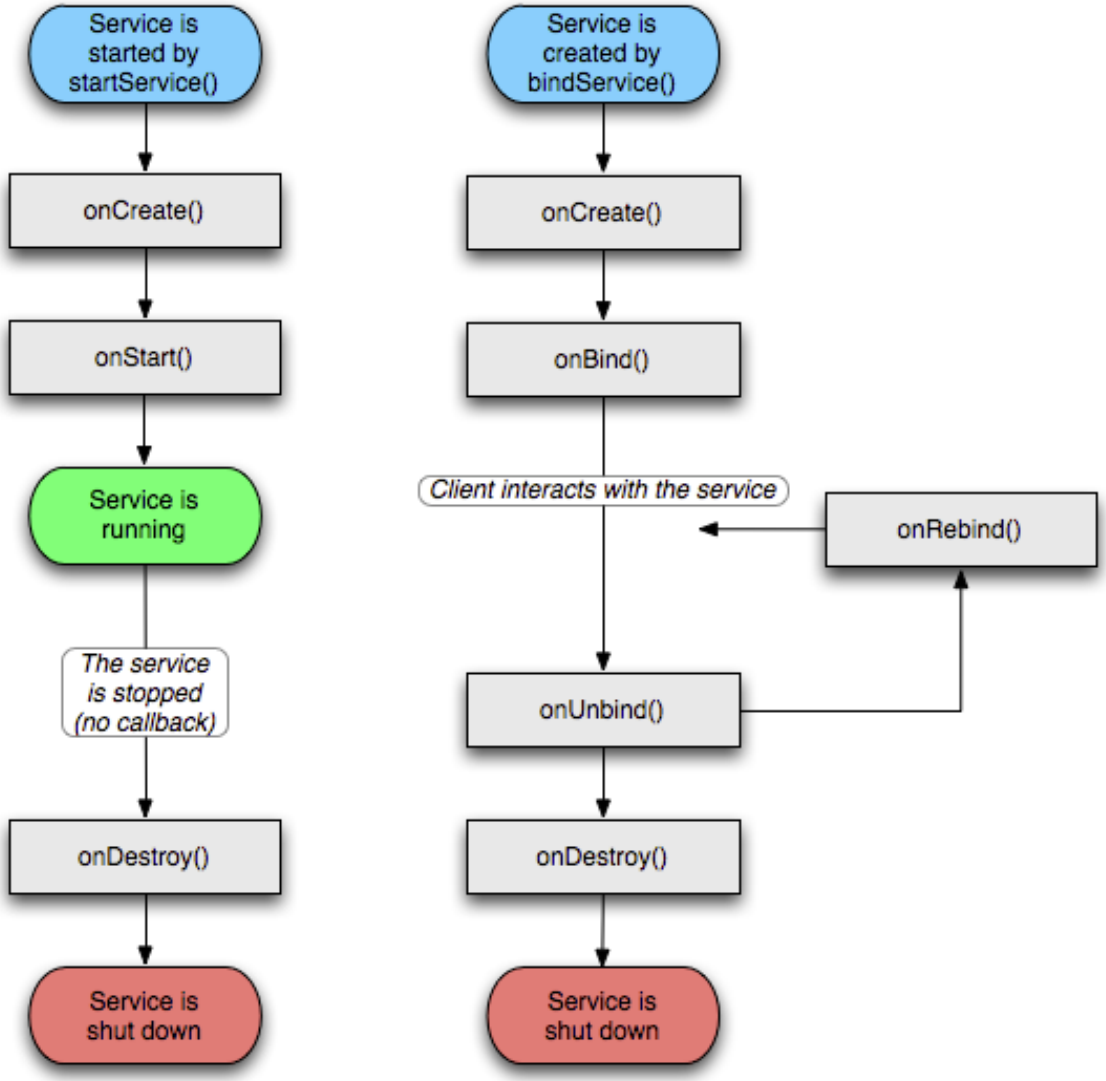
SERVICE PRIMITIVES

- Service activation, here repetition is allowed:
  - startService(intent);
- Service binding:
  - bindService(intent, serviceConnection, Context.BIND_AUTO_CREATE);
- Service deactivation:
  - stopService(intent);
- Service unbinding, repetition is not allowed:
  - unbindService(serviceConnection);

SERVICE DEACTIVATION

- A service is destroyed itself by calling **stopSelf**() if it is not bound; or calling **stopService**() or **unbindService**() or both, if for *unbind* it is the last one to unbind from the service.
- **Note**: any component, including activities, can be finished by Android at any time and the execution of **onDestroy**() is NOT assured.

SERVICE CYCLE OF LIFE



LIFECYCLE OF A BOUND SERVICE

- If the service is started and bounded in any order, it will remain active (it's not destroyed) until it is stopped and unbounded.
- The onUnbind() method is called only for the last
- The onRebind() method is called only for the first

